

Insertion Sort

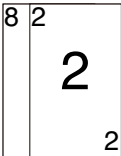
插入排序

本页面将简要介绍插入排序。

定义

插入排序（英语：Insertion sort）是一种简单直观的排序算法。它的工作原理为将待排列元素划分为「已排序」和「未排序」两部分，每次从「未排序的」元素中选择一个插入到「已排序的」元素中的正确位置。

一个与插入排序相同的操作是打扑克牌时，从牌桌上抓一张牌，按牌面大小插到手牌后，再抓下一张牌。



性质

稳定性

插入排序是一种稳定的排序算法。

时间复杂度

插入排序的最优时间复杂度为 $O(n)$ ，在数列几乎有序时效率很高。

插入排序的最坏时间复杂度和平均时间复杂度都为 $O(n^2)$ 。

代码实现

伪代码

 
C++Python

```
1 void insertion_sort(int arr[], int len) {
2   for (int i = 1; i < len; ++i) {
3     int key = arr[i];
4     int j = i - 1;
5     while (j >= 0 && arr[j] > key) {
6       arr[j + 1] = arr[j];
7       j--;
8     }
9     arr[j + 1] = key;
10  }
11 }
```

```
1 def insertion_sort(arr, n):
2   for i in range(1, n):
3     key = arr[i]
4     j = i - 1
5     while j >= 0 and arr[j] > key:
6       arr[j + 1] = arr[j]
7       j = j - 1
8     arr[j + 1] = key
```

折半插入排序

插入排序还可以通过二分算法优化性能，在排序元素数量较多时优化的效果比较明显。

时间复杂度

折半插入排序与直接插入排序的基本思想是一致的，折半插入排序仅对插入排序时间复杂度中的常数进行了优化，所以优化后的时间复杂度仍然不变。

代码实现

 **C++**

```
1 void insertion_sort(int arr[], int len) {
2     if (len < 2) return;
3     for (int i = 1; i != len; ++i) {
4         int key = arr[i];
5         auto index = upper_bound(arr, arr + i, key) - arr;
6         // 使用 memmove 移动元素，比使用 for 循环速度更快，时间复杂度仍为 O(n)
7         memmove(arr + index + 1, arr + index, (i - index) * sizeof(int));
8         arr[index] = key;
9     }
10 }
```

本页面最近更新：2024/11/10 20:22:04, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[NachtgeistW](#), [AprilNEA](#), [clee01](#), [countercurrent-time](#), [EmptyDreams](#), [Enter-tainer](#), [Great-designer](#), [H-J-Granger](#), [hsfzLZH1](#), [iamtwz](#), [Junyan721113](#), [Konano](#), [mcendu](#), [Menci](#), [ouuan](#), [partychicken](#), [shawlleeyw](#), [SukkaW](#), [thy233](#), [Tiphereth-A](#), [TrickEye](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用